

Full-Stack Developer Take-Home Exercise

Estimated time: 12 to 18 hours

Deadline: 3 business days from when you receive this email

Submit: A live public URL + GitHub repo + demo video to careers@rovedashcam.com

What we want to see

We want to see how you build a real internal product end to end. This assignment is intentionally broader than a one-day exercise because we care about:

- How you structure a real multi-screen app with two different audiences (internal staff and external candidates)
- How you design flows, not just features
- How you ship something polished, not just functional
- How you handle authentication, file uploads, and document generation
- How you make taste-driven decisions when we leave things open

You will build a small internal recruitment tool for ROVE. We will open the URL you send us and use it as a real HR user would.

The brief

Build a web app called ROVE Hire. It is the internal tool the ROVE HR team uses to manage candidates from the moment a resume arrives to the moment an offer letter is generated. The hiring manager and HR person should be able to do everything they need without leaving this app or copying data into separate tools.

The product team will demo your build during evaluation. The experience matters as much as the code.

Required features

1. HR sign-in and dashboard (the home view)

- HR users must sign in before they can do anything. Method is your choice (email + password, Google, magic link, whatever fits best).
- Once signed in, they land on a dashboard that lists every candidate in the pipeline.
- Each row shows: candidate name, role they applied for, current status, last activity date.
- Status values: Applied, Form Submitted, Interview Scheduled, Offer Sent, Hired, Rejected.
- The dashboard supports filtering by status and searching by name or role.
- A candidate's row click opens that candidate's full profile (see feature 6).

2. Create a job opening

- HR can create a new job opening with: title, description (rich text or markdown is fine), required skills (tags), and status (Open or Closed).
- A list view shows all job openings. Each opening shows the number of candidates currently associated with it.
- Closed openings are still visible, but cannot have new candidates added.

3. Add a candidate manually

- HR clicks "+ Add Candidate" and:
 - Uploads the candidate's resume (PDF, max 10 MB).
 - Picks the job opening to associate the candidate with.
 - Enters the candidate's name and email.
- On submit, the system:
 - Saves the resume to a backend so it can be downloaded later.
 - Creates a candidate record in status Applied.
 - Generates a secure magic link for the candidate to complete their application (see feature 4).
 - Surfaces the magic link in the UI so HR can copy it (you do not need to actually send an email; the link being copyable from the screen is enough).

4. Public candidate application page (no login)

- The magic link from feature 3 opens a public page (no sign-in) where the candidate completes their basic information.
- Form fields the candidate fills in: phone number, current location, current role, notice period, salary expectation, LinkedIn URL.
- Magic-link tokens expire in 14 days. Show a clean "this link has expired" screen if used after expiration.
- One-time-use: after the candidate submits, the link should no longer accept submissions.
- On successful submit, the candidate's status moves to Form Submitted, and they see a friendly confirmation screen.

5. Schedule an interview

- From the candidate's profile, HR can schedule an interview: date, time, interview type (Screening or Technical), interviewer name, optional notes.
- Scheduled interviews appear on the candidate's profile timeline and on a separate Interviews page sorted by date.
- After an interview is scheduled, the candidate's status moves to Interview Scheduled.
- After an interview, HR can mark the interview as "Completed" and record short feedback: a hire / no-hire / maybe recommendation plus a free-text note. Feedback is visible on the candidate's profile.

6. Candidate profile page (the spine of the app)

- Every candidate has a profile page that shows:
 - Basic info (name, email, phone, location, role, etc.)
 - Resume download link
 - Status with an obvious visual cue
 - Timeline of events (applied, form submitted, interview scheduled, feedback recorded, offer sent, etc.) ordered most recent first
 - Action buttons that change based on the current status (e.g., "Move to Offer" only appears after a completed interview)

7. Generate offer letter and NDA (the core technical step)

This is the centerpiece of the assignment. From a candidate's profile in status Interview Scheduled or later, HR can click "Generate Offer Documents". They:

- Fill in a small form: role title, salary (currency + amount), start date, reporting manager name, location.
- Click Generate.
- The system produces two downloadable PDFs:
 - Offer Letter with the candidate's name, role, salary, start date, manager, and location auto-filled into a template you design.
 - NDA with the candidate's name, today's date, and a basic confidentiality clause from a template you design.
- The PDFs are stored on the backend (so they can be re-downloaded later) and shown as downloadable files on the candidate's profile.
- Once generated, the candidate's status moves to Offer Sent.

You design both templates yourself. They should look like real documents, not lorem ipsum. Use a real layout (header, body, signature blocks). The templates do not need to be legally accurate; they need to look professional enough that we would not be embarrassed to send them.

8. Mark as hired or rejected

- From any non-terminal status, HR can mark a candidate as Hired or Rejected.
- Hired requires an actual offer to have been generated first.
- Rejected requires a short reason (free text).
- Both actions are logged in the timeline.

Hard requirements (technical)

- Frontend: Next.js. App Router preferred but Pages Router is acceptable.
- Backend: Any language and framework. The backend can be Next.js API routes, a separate service, or a backend-as-a-service. Your call.

- **Database:** Your choice. Explain why you picked it in the README (two sentences is fine).
- **File storage:** Your choice. Resumes and generated PDFs must be persisted and re-downloadable.
- **Hosting:** Deploy the full stack to a public URL that we can open in our browser. We will NOT run it locally. Send us the link.
- **Design quality:** Pixel-perfect, polished, and feels like a real internal tool. Think Linear or Notion-level care, not bootstrap-default. Treat this like a real product, not a homework.
- **UX judgment:** We gave you the feature list, not the design. Make every screen feel intentional. Make navigation obvious. Make loading, empty, and error states feel handled, not afterthoughts. Use your best taste anywhere we left a call open, and tell us about your choices in the README.
- **Persistence is real:** refreshing the page or signing out and back in should not lose any data. Resumes and generated PDFs must still be downloadable later.

What we are NOT looking for

- Real email delivery (showing the magic link in the UI so HR can copy it is enough).
- Real e-signature integration on the PDFs.
- Real Google Workspace account provisioning.
- Real Monday.com or third-party integration.
- Production-grade scaling, observability, or monitoring.

Submitting

Email your submission to careers@rovedashcam.com with the subject:

Assignment Submission - <Your Name> - Full-Stack Developer

Include:

1. The live URL where we can open and use your app.
2. A link to your GitHub/GitLab repository with the code (if private, add careers@rovedashcam.com as a collaborator).
3. Test HR login credentials so we can sign in. Pre-populate the system with at least 3 sample job openings, 5 sample candidates in varying states (one in Applied, one in Form Submitted, one in Interview Scheduled with feedback, one in Offer Sent with downloadable PDFs already generated, one Rejected). We want to see the populated experience immediately, not have to click our way through to populate it.
4. A demo video walkthrough (3 to 5 minutes). Record your screen as you go through each flow: HR sign-in, dashboard, create a job opening, add a candidate, copy the magic link, switch to a private browser tab and complete the candidate form, return to HR view, schedule an interview, mark feedback, generate offer letter and NDA PDFs, mark the candidate as hired. A simple Loom or QuickTime

screen recording is fine. Voiceover is appreciated but optional; if no voiceover, add brief on-screen captions for each section. The goal is for us to see the app in action even before we open the live URL.

5. A README in the repo covering:

- How it is hosted (one or two sentences, no need to recommend services).
- Tech stack and why. What you chose for backend, database, file storage, hosting, and why each was the right call for this problem.
- How you approached PDF generation. The library or technique you used, why, and what you would change at scale.
- What you would do next. If you had another two days, what would you build or improve? Anything you cut for time?
- Anything that broke or felt off. Tell us what you would not put in production yet and why.

A few last notes

- This is meant to be a few days of focused work, not a marathon. If you find yourself going past 18 hours, stop, submit what you have, and tell us in the README what you would have done next.
- Use AI tools (Copilot, ChatGPT, Claude) however you normally would. We use them too. Since AI takes a lot of the boilerplate off your plate, spend the saved time on the UX, the design, and the details. We want to see your taste and judgment, not just code that runs. We will ask you about the choices you made and why in the next round.
- Questions are welcome. Reply to the email any time and a recruiter will get back to you.

We are looking forward to seeing what you build.

Team ROVE